EXAM RULES OF THUMB

- Compute the output yourself first. "None of the above" was right 4 of 23 times on the sample (Q3, 7, 8, 9).
- `print(a, b)` space-separates. Only the last expression in a cell auto-displays.

PYTHON

- Types: int, float, bool (`True`/`False`), str, list, dict. `type("250")` is **str**.
- `/` always float (`16/2`=8.0). `//` floors (`-5//2`=-3). `%` remainder (`18%5`=3). `**` power.
- `==` compares, `=` assigns. Case-sensitive: `"Py"=="py"` False. Precedence: not, and, or.
- Str methods return a NEW str: `.upper() .lower() .title() .count("l") .replace("o","u")`. `f"{a} {b}"`.
- List: index from 0, `-1` is last. `[a:b]` **stop excluded**: `[:3]` `[1:]` `[1:3]`.
- `append(x)` end, `insert(i,x)`, `remove(val)`, `pop(i)` returns item (default last), `del l[i]`.
- `l.sort()` in place, returns **None**. `sort(reverse=True)`. `sorted(l)` returns new.
- `y = x` is an **alias** (shared). `y = x.copy()` or `x[:]` is independent (Q1).
- Dict `{k:v}`: `d[k]`, `.keys() .values() .items()`, `d[k]=v` adds, `.update({})`, `.pop(k)`, `del d[k]`, `.clear()`.
- `if / elif / else`: only the first True branch runs. `x in list`.
- Loops: `for x in l`; `for i, x in enumerate(l)`; `for k, v in d.items()`.
- `range(stop)`, `range(start, stop, step)`: **stop excluded**.
- `def f(a): return a+1`. No return gives `None`. Built-ins: `len max min sum round type`.

PANDAS: BUILD AND INSPECT

- `import pandas as pd`, `import numpy as np`.
- DataFrame from list of lists: **inner list = ROW**. From dict: **key = COLUMN**. Also `index=`, `columns=`, `pd.read_csv()`.
- `.shape` (rows, cols). `.columns` labels. `.index`. `.dtypes`. `len(df)` = rows. `max(df)` = a column NAME.
- `.head(n)` `.tail(n)` (default 5). `.info()` shows non-null counts. `.describe()` **numeric columns only**.
- `df['c']` Series. `df[['c']]` DataFrame. `df[['a','b']]` DataFrame, your order. `df.c` fails with spaces (SyntaxError).
- New column: `df['c'] = scalar` or array (length = rows). `np.random.randint(lo, hi, size=n)` **hi excluded**. `np.arange(n)`.
- `df.assign(a=.., b=..)` returns a NEW df. `df.insert(pos, 'name', vals)` in place.
- Column math is elementwise: `(df.p1 - df.p0) / df.p0 * 100`.
- Series stats: `sum count mean median std max min describe`.
- `value_counts()` sorts descending, skips NaN; `normalize=True` gives proportions.

PANDAS: MISSING, SORT, INDEX

- `isna()` mask. `isna().sum()` = missing. `count()` = non-NaN. `len` = all rows. count + missing = len (Q5: 5 and 3).
- `fillna(0)` returns new: `df['c'] = df['c'].fillna(0)`.
- `sort_values('c', ascending=False)`; two columns: `by=['a','b'], ascending=[True, False]`.
- `inplace=True` returns **None**. Never `df = df.sort_values(..., inplace=True)`.
- `set_index('c')` moves column to index, so columns drop by 1 (Q19). `reset_index()` undoes. `sort_index()`.
- `rename(columns={'old':'new'})`, `rename(index={...})`. Unlisted labels stay (Q17). Reassign or `inplace=True`.
- Key: `df['Y'].astype(str) + '_' + df['T']`.

FILTERING

- `df[df['c'] == 'x']` keeps matching ROWS only. Columns unchanged: shape (n, same cols) (Q6).
- `&` `|` `~`, **each condition in parentheses**: `df[(A) & (B)]`. `and`/`or` gives ValueError. No parentheses gives TypeError. `A & (B | C)` is not `(A & B) | C` (Q20).
- `.str.startswith('A')`, `.isin([..])`, `~` for not in. `df.query("c > 5 and s == 'x'")`, `@var` for variables.
- `np.where(cond, 'Big', 'Small')` returns an array: `df['t'] = np.where(...)`.

DUPLICATES, UNIQUE, SAMPLE

- `duplicated('c')`: True for each **repeat** (first stays False). `keep='last'`. `keep=False` marks all copies.
- `df[~df.duplicated('c')]` = first per value. Same as `drop_duplicates('c')`. Appear once only: `~duplicated('c', keep=False)`.
- Smallest per group: sort ascending by [group, size], then `drop_duplicates('group')`. `keep='last'` gives the biggest.
- `unique()` array, includes NaN. `nunique()` counts, ignores NaN. `sample(n)`, `sample(frac=0.1)`.

GROUPBY

- `g = df.groupby('S')` is lazy. `len(g)` = number of groups. `g.groups.keys()`. `g.get_group('x')`.
- Output sorted by key; key becomes the **index**.
- `g.mean(numeric_only=True)` (text columns raise TypeError). `g['c'].mean()` Series. `g[['a','b']].mean()` DataFrame.
- `size()` = rows incl NaN. `count()` = non-NaN. Q3: size [2,3], count [2,1].
- `agg(['min','max'])`: columns named **min, max**; shape (n_groups, 2) (Q12).
- `agg({'a':'mean','b':'max'})`: columns keep **a, b** (Q4).
- `groupby([a,b])['c'].mean()` has a MultiIndex. `.unstack()` moves the inner level to columns. `pd.pivot_table(df, index=, columns=, values=, aggfunc=)`.
- `transform('mean')` returns same length as df. `idxmax()` returns the index **label**; then `df.loc[labels]`.

TRAPS

- Shape is (rows, cols). Filter changes rows; `set_index` removes a column.
- NaN: `count` skips it; `size`, `len` do not; `nunique` skips it; `describe` is numeric only.
- Off by one: slices, `range`, `randint` all exclude the stop value.
- `.columns` = labels, not data. `.copy()` independent. `inplace` returns None.